

# Hash Functions

While encryption is about *hiding* data, a **hash function** is about *verifying* data.

Think of a cryptographic hash function like a digital blender. You throw data into it (a word, a password, or an entire movie file), turn it on, and it spits out a unique, fixed-size string of characters called a **hash value** (or a digital fingerprint).

The most important thing to remember is that hashing is a **one-way street**. You can easily turn a file into a hash, but you can *never* reverse a hash back into the original file.

## The Four Rules of a Great Hash Function

To be useful in security, a hash function (like **SHA-256**, the industry standard) must follow four strict mathematical rules:

### 1. It is Deterministic

If you input the word "Apple" today, tomorrow, or ten years from now, it will **always** spit out the exact same hash value.

### 2. It has a Fixed Output Size

No matter how big or small the input is, the output hash is always the exact same length.

- Input a single letter a → 64-character hash.
- Input the entire text of *War and Peace* → 64-character hash.

### 3. It has an "Avalanche Effect"

If you change even a single pixel in an image or one lowercase letter in a 500-page book, the resulting hash changes completely. There is no pattern to the change, making it impossible to guess what was altered.

### 4. It resists "Collisions"

A collision happens when two completely different inputs produce the exact same hash. A good cryptographic hash function makes this practically impossible. Finding a collision in SHA-256 is roughly equivalent to winning the powerball lottery every single day for a century.

## Hashing in Action: The Real-World Examples

Because hashes act as unique, unforgeable fingerprints, we use them constantly behind the scenes.

### 1. How Websites Secure Your Passwords

Websites should **never** store your actual password in their databases. If a hacker steals the database, they get everyone's credentials.

Instead, when you create an account, the server hashes your password and stores *only the hash*.

- You type: P@ssword123
- Server stores:  
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e

When you log in later, the server hashes your input again. If the new hash matches the stored hash, you are let in. If a hacker steals the database, all they get is a list of useless hashes that they cannot reverse-engineer back into actual passwords.

## **2. File Integrity (Checking for Tampering)**

When you download a software update, how do you know a hacker didn't intercept the download and inject malware into it?

Developers will publish the official hash of the file on their website. Once downloaded, your computer automatically hashes the file on your hard drive. If your computer's hash matches the developer's website hash perfectly, you know the file is 100% authentic and hasn't been altered by a single byte.

## **3. The Blockchain (Bitcoin)**

Hashes are the literal chains that link blocks together in a blockchain. Each block contains the hash of the *previous* block. Because of the avalanche effect, if someone tries to alter a transaction from five years ago, it changes that block's hash, which breaks the chain all the way down the line, alerting the network instantly to the fraud.